

BusinessSAAS

Production Checklist Reference

Backend · Frontend

Gap Analysis · Priority Roadmap · June 2026

This document is the internal reference for the BusinessSAAS project. It captures what is already production-ready, what is missing before go-live, and what should be added within the first 3–6 months after launch. Use it to avoid retrofitting decisions that are painful to add later.

Status Legend

[DONE]	Implemented and production-ready	[PARTIAL]	Exists but incomplete or inconsistent	[MISSING]	Not implemented yet
--------	----------------------------------	-----------	---------------------------------------	-----------	---------------------

Part 1 — Backend (Go / Fiber v3 / PostgreSQL)

Current state: Auth, RBAC, CRM, Tasks, Platform layer, API docs — all complete.

Category	Status	Gap / Note
Clean Architecture (H→S→R)	[DONE]	—
JWT Auth + Refresh Tokens	[DONE]	—
RBAC — Roles, Permissions	[DONE]	—
Multi-tenancy (org_id isolation)	[DONE]	—
Rate limiting	[PARTIAL]	Auth endpoints only — no global limit
Audit logging	[PARTIAL]	Security events covered; business events incomplete
API documentation (OpenAPI 3.0)	[DONE]	—
CI/CD (GitHub Actions)	[DONE]	—
Docker Compose (4 containers)	[DONE]	—
Database migrations (Goose)	[DONE]	—
Unit + Integration tests	[DONE]	—
Security headers middleware	[MISSING]	X-Frame-Options, HSTS, CSP, etc. not added
Global rate limiting	[MISSING]	No per-IP limit outside auth routes
Request body size limit	[MISSING]	No 10 MB cap — memory exhaustion risk
Idempotency keys	[MISSING]	Critical mutations (convert, create org) can duplicate
Request timeout middleware	[MISSING]	Slow DB query can hold goroutine indefinitely
Prometheus metrics	[MISSING]	No observability — can't debug production issues
OpenTelemetry tracing	[MISSING]	No distributed trace across services
Slow query logging (>500ms)	[MISSING]	No DB query instrumentation
PostgreSQL pool tuning	[PARTIAL]	Default pool config — not tuned for load
Consistent error wrapping (%w)	[PARTIAL]	Some services missing fmt.Errorf context
golangci-lint config	[MISSING]	No .golangci.yml — code quality not enforced in CI
Background job worker	[MISSING]	No queue for email, webhooks, async tasks
Pagination on all list endpoints	[PARTIAL]	Some CRM list endpoints missing pagination
Feature flags (per-org)	[MISSING]	Runtime config change not possible
Soft delete consistency	[PARTIAL]	Not all entities have deleted_at
API versioning strategy (v2)	[MISSING]	No plan for breaking changes or Sunset headers
GDPR hooks	[MISSING]	No data export or right-to-erasure endpoints

Backend — Priority Roadmap

PRIORITY 1 — Before going to production (1–2 weeks)

- Security headers middleware — X-Content-Type-Options, X-Frame-Options, HSTS, Referrer-Policy, Permissions-Policy
- Global rate limiting — per-IP limit on all API routes (not just /auth)
- Request body size limit — 10 MB cap to prevent memory exhaustion attacks
- Idempotency keys middleware — Redis-backed, 24h TTL, scoped to user+org+path for critical mutations
- Request timeout middleware — 30s default, 60s for report endpoints
- golangci-lint config (.golangci.yml) + enforce in CI pipeline

PRIORITY 2 — First 3 months after launch

- Prometheus metrics — HTTP request counts, latencies, DB query durations, auth failure counters
- Slow query logging — trace and log all DB queries exceeding 500ms
- PostgreSQL connection pool tuning — MaxConns=25, MinConns=5, MaxConnLifetime=30m
- Consistent error wrapping — `fmt.Errorf("package.Method: %w", err)` everywhere in service layer
- Background job worker — PostgreSQL-backed queue for email, webhooks, async report generation
- Pagination on all remaining list endpoints — standard `{ items, meta: { page, limit, total, totalPages } }`
- OpenTelemetry tracing — distributed traces across HTTP → service → DB layers

PRIORITY 3 — 6 months / scaling phase

- Feature flags (per-org, DB-backed) — enable/disable features per organization without deploy
- Soft delete consistency — `deleted_at` column on ALL entity tables, enforced in all list queries
- API versioning strategy — plan for /api/v2, add Deprecation + Sunset headers to v1 endpoints
- GDPR hooks — data export endpoint (JSON/CSV) and right-to-erasure endpoint for user data
- Read replica support pattern — route read-heavy report queries to replica
- Load testing — baseline performance benchmarks (k6 or vegeta) before scaling

Backend Timeline Summary

Now (1–2 wks)	Security headers • Global rate limit • Request size limit • Idempotency keys • Timeout middleware • golangci-lint CI
Month 1–3	Prometheus metrics • Slow query logging • Pool tuning • Error wrapping • Background jobs • Pagination
Month 4–6	Feature flags • Soft deletes • API v2 strategy • GDPR hooks • Read replica • Load testing

Part 2 — Frontend (Next.js App Router / TypeScript / Zustand)

Current state: Steps 1–4 complete — Auth, AuthProvider, org flows. Step 5 (Dashboard shell) is next.

Category	Status	Gap / Note
TypeScript strict mode	[DONE]	—
Axios + interceptors (401→refresh)	[DONE]	—
Zustand stores (auth, perms, ui)	[DONE]	—
React Hook Form + Zod (forms)	[DONE]	—
next-themes (dark/light)	[DONE]	—
Access token in memory (not storage)	[DONE]	—
usePermission hook	[DONE]	—
GSAP animations	[DONE]	—
AuthProvider + silent refresh	[DONE]	—
Server state (TanStack Query)	[MISSING]	CRITICAL — impossibly hard to retrofit after 20+ pages
URL state for filters/search/pages	[MISSING]	CRITICAL — retrofit requires rewriting all list pages
Error Boundary	[MISSING]	Any uncaught React error = blank white screen
Permission Gate component	[MISSING]	can() spread inline across 50+ components
Toast notification system	[MISSING]	No user feedback on mutations
Skeleton loading state pattern	[MISSING]	No standard — spinner or nothing currently
Zod API response validation	[MISSING]	Backend shape mismatch = silent crash
Empty state components	[MISSING]	New org first-load shows nothing
Error state components	[MISSING]	Fetch failure has no designed state
Unsaved changes warning	[MISSING]	Form data lost on navigation
Accessibility — ARIA, keyboard nav	[MISSING]	Hard to retrofit; should be built in from start
CSP headers (next.config)	[MISSING]	XSS attack vector open
Font optimization (next/font)	[PARTIAL]	Fonts planned but not loaded via next/font
Component testing (Vitest + RTL)	[MISSING]	Writing tests is easy alongside; retroactive is hard
E2E testing (Playwright)	[MISSING]	Auth flow has no automated coverage
Bundle size monitoring	[MISSING]	Bundle silently grows without alert
Virtualization (TanStack Virtual)	[MISSING]	500+ row list will freeze the browser
Error tracking (Sentry)	[MISSING]	Production crashes unknown without it
Storybook	[MISSING]	Component isolation and documentation
Web vitals / Lighthouse CI	[MISSING]	Performance regressions go undetected

Frontend — Priority Roadmap

Two decisions that are structural — they must be made BEFORE building more pages. Retrofitting these after 20+ pages is a full rewrite of all data-fetching and list-filter logic.

CRITICAL — Add before building Step 5 (can never be easily retrofitted)

- TanStack Query (@tanstack/react-query) — server state management: caching, background refetch, cache invalidation, optimistic updates. Zustand stores only user/auth/UI state, never API data.
- URL state for filters, search, pagination (nuqs library) — all list page filters live in the URL so browser back/forward works, pages are shareable, and refresh preserves state.
- Error Boundary — wrap root layout and dashboard layout; prevents blank white screen on any uncaught error.
- Permission Gate component (wrapper) — replaces scattered inline can() checks with a single reusable component.
- Toast notification system (sonner) — centralized success/error feedback for all mutations.
- Skeleton loading state pattern — establish one skeleton template early; all pages copy the pattern.

Install: `bun add @tanstack/react-query nuqs sonner @tanstack/react-query-devtools`

PRIORITY 1 — Apply to every page as it is built

- Zod schema for every API response — validate shape on the way in, not just on form submission
- Skeleton component matching the actual layout of each page (not a generic spinner)
- Empty state component — designed screen when a list has zero items (new org, first lead, etc.)
- Error state component — designed screen when a fetch fails, with a retry action
- Unsaved changes warning — useUnsavedChanges(isDirty) hook on every form page
- Accessibility basics — semantic HTML elements, aria-label on all icon buttons, aria-invalid + role=alert on form errors, Escape key closes all modals

PRIORITY 2 — First 3 months

- CSP headers in next.config — Content-Security-Policy, X-Frame-Options, Referrer-Policy, Permissions-Policy, X-Content-Type-Options
- Font optimization via next/font — load Syne and Inter/DM Sans through Next.js to eliminate layout shift and external font requests
- Component testing setup (Vitest + React Testing Library) — write tests alongside components, not after the fact
- E2E testing (Playwright) — cover at minimum: login, token refresh, org switch, logout, and one permission-gated action
- Bundle size monitoring (@next/bundle-analyzer) — run after every major feature; target First Load JS < 250 KB per route

PRIORITY 3 — Scaling phase

- Virtualization (TanStack Virtual) — required for any list that can exceed 200 rows (leads, contacts, deals, activities)
- Error tracking (Sentry) — production crash reporting with session replay; mask all PII in config before enabling
- Storybook — component isolation, visual regression testing, design system documentation
- Lighthouse CI — automated performance, accessibility, and SEO scoring on every PR
- Web vitals dashboard — track LCP, CLS, FID/INP in production via Next.js analytics or a custom endpoint

Frontend Timeline Summary

Before Step 5

TanStack Query • URL state (nuqs) • Error Boundary • Permission Gate • Toast (sonner) • Skeleton pattern

Each page	Zod API validation • Skeleton component • Empty state • Error state • Unsaved warning • Accessibility basics
Month 1–3	CSP headers • next/font • Vitest + RTL • Playwright E2E • Bundle monitoring
Scaling	TanStack Virtual • Sentry • Storybook • Lighthouse CI • Web vitals dashboard

Keep this document updated as items are completed. Mark [DONE] in your own copy when each item is implemented. The backend Priority 1 items and the frontend TanStack Query + URL state decisions are the only items that cannot safely wait.